Orkut Sphere: Scalable Social network Analysis

Taha Mirza
Master's in Applied Computer Science
Concordia University
taha.mirza@mail.concordia.ca

Aafreen Fathima Anwar Basha
Master's in Applied Computer Science
Concordia University
a_anwarb@live.concordia.ca

Syeda Rahmathunnisa Fatima
Master's in Applied Computer Science
Concordia University
sy_fat@live.conordia.ca

Shariq Anwar
Master's in Applied Computer Science
Concordia University
a_shariq@live.concordia.ca

Finn Kleckner
Master's in Applied Computer Science
Concordia University
f_kleckn@live.concordia.ca

ABSTRACT

This work presents a distributed graph analytics system built on NebulaGraph to process the large-scale Orkut social network dataset, which contains over three million users, 117 million friendship edges, and five thousand communities. The system demonstrates how very large social graphs can be efficiently ingested, stored, and analyzed within a distributed environment.

A multi-node NebulaGraph cluster is deployed using Docker, incorporating sharding, replication, and the Raft consensus protocol to ensure scalability and fault tolerance. High-volume data is ingested through python query for users, friendships, and community memberships. The schema accommodates future extensibility through interaction edges into distinct spaces to maintain long-term performance.

The platform enables analytical tasks such as community-based recommendations, influential user detection, organically emerging community identification, and estimation of small-world properties. Observations highlight the impact of partitioning strategies and leader balancing on system performance.

Overall, the system illustrates NebulaGraph's suitability for large- scale social graph analysis and provides an extensible foundation for research on community dynamics and influence propagation.

PVLDB Reference Format:
Taha Mirza, Aafreen Fathima Anwar Basha, Syeda Rahmathunnisa Fatima, Shariq Anwar, and Finn Kleckner. OrkutSphere: Scalable Social network Analysis. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at URL_TO_YOUR_ARTIFACTS.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097. doi:XX.XX/XXX.XX

1. INTRODUCTION

Large-scale social networks are essential for understanding how communities form, how information spreads, and how influence flows across populations. Foundational work in social network analysis has shown that structural properties such as clustering, centrality, and path length play a critical role in understanding human relationships and organizational behaviour [1]. Later studies on complex networks further demonstrated that large real-world graphs exhibit small-world characteristics, heavy-tailed degree distributions, and tightly knit community structures, all of which demand specialized computational systems for efficient analysis [2].

Traditional relational databases are not optimized for graph-structured workloads, as multi-hop traversals and repeated joins introduce significant overhead. Modern graph query languages and graph database systems address these limitations, offering traversal-centric query operators and optimized storage structures [3]. Distributed graph engines, in particular, are designed to support large-scale data processing through partitioning, replication, and parallel execution across multiple nodes [5,6].

The Orkut social network dataset provides a large real-world benchmark for evaluating such systems. The dataset consists of over three million users, 117 million friendship edges, and five thousand ground-truth communities derived from user-created groups. It has been widely used to study the reliability of community detection methods and the nature of social cohesion in online platforms [4] due to its size, density, and rich community structure.

NebulaGraph is a distributed, RAFT-based graph database designed for scalability, fault tolerance, and high-throughput analytical workloads [6]. Its storage engine uses a shared-nothing architecture where vertices and edges are sharded across partitions, each replicated across multiple storage nodes. Leader election, heartbeat monitoring, and log replication via RAFT ensure strong consistency and rapid recovery from node failures. These characteristics make NebulaGraph an appropriate platform for large-scale social network analytics.

This project builds a distributed analytics platform for the Orkut graph using NebulaGraph. The system includes:

(1) a multi-node cluster deployed through Docker, supporting sharding and replication;

(2) high-throughput ingestion pipelines using insert operations, connection pooling, and parallel Python loaders;

(3) analytical queries involving community recommendation, influential user identification, organic community growth, and small- world distance estimation.

Experimental evaluations highlight the role of partitioning strategies, cluster scale-out, and leader balancing in enabling efficient distributed queries on hundred-million-edge graphs.

2. BACKGROUND

2.1. NebulaGraph Architecture

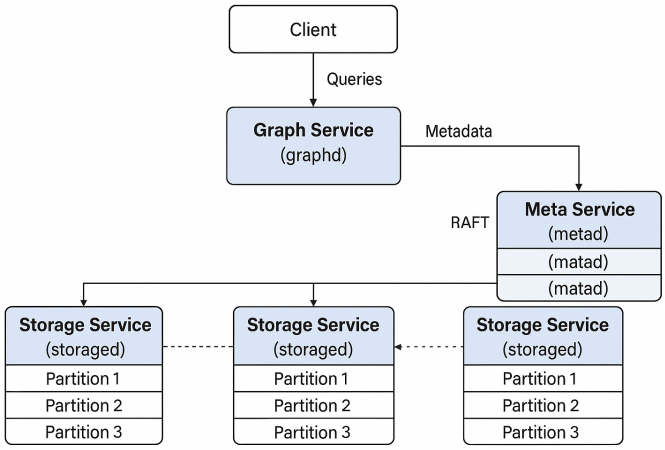
NebulaGraph is a distributed, RAFT-based graph database designed for large-scale graph storage and analytics. Its architecture separates responsibility into three core services:

(i) *Meta Service (metad)*, which stores metadata such as schema definitions, partition placement, and cluster topology;

(ii) *Storage Service (storaged)*, which stores vertices and edges in partitioned form; and

(iii) *Graph Service (graphd)*, which receives queries from clients and routes them to the appropriate storage nodes.

This separation allows the system to achieve high availability and efficient parallel query execution across many machines.



NeburlaGraph Distributed Architecture

2.2. Docker-Based Multi-Node Deployment

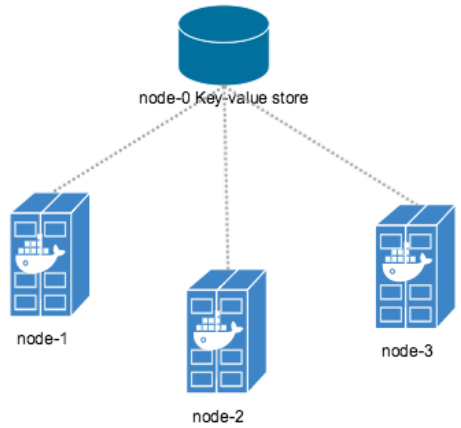
To emulate a distributed cluster, NebulaGraph services are deployed using Docker containers. The configuration includes **7 containers**:

- **3 Meta nodes** to ensure metadata replication and robust cluster coordination,
- **5 Storage nodes** to hold replicated graph partitions, and
- **2 Graph node** to serve as the client-facing endpoint for query execution.

Docker provides isolation, reproducibility, and simplified scaling, making it suitable for experimentation with distributed database behaviours.

2.3. Partitioning and Data Sharding

NebulaGraph horizontally partitions the dataset into **N partitions** across multiple storage nodes. Each vertex is assigned to a partition based on hashing (typically VID % partition_num).Sharding serves



twopurposes:

(1) **Parallelism:** Queries traverse multiple partitions concurrently, improving throughput.

(2) **Load distribution:** Graph data and query workloads are spread evenly across the cluster.

In the demonstration environment, the dataset is divided into **3 partitions**, which are distributed across all storage nodes.

2.4. Replication and High Availability

To tolerate failures, NebulaGraph maintains **replication at the partition level**. Each partition has **three replicas**, one stored on each of the three storage nodes. RAFT consensus governs consistency across these replicas by designating:

- **One leader replica** (responsible for writes), and
- **Two follower replicas** (kept synchronized through log replication).

If the leader becomes unavailable, a follower automatically assumes leadership. This enables continuous query execution even during node failures.

2.5. Fault Tolerance Through RAFT Consensus

RAFT provides NebulaGraph with strong consistency guarantees and robust failure recovery. It continuously monitors node health via heartbeats and triggers leader elections when necessary.

This ensures that:

- A storage node can go offline without affecting availability,
- Writes remain consistent across replicas, and
- The cluster maintains correct behaviour under partial failure.

This feature is central to the demonstration, where intentionally stopping a storage node does not interrupt running Python ingestion scripts.

2.6. Automated Data Ingestion Pipeline

The system ingests a **~1.7 GB social graph**, consisting of millions of vertices and edges. Instead of manual insertion, ingestion is automated through Python scripts that use Nebula's client API. Core aspects include:

- **Batch inserts** to reduce network overhead,
- **Connection pooling** to reuse client sessions efficiently,

This design reflects realistic ingestion workloads seen in production-scale graph systems.

3. SYSTEM DESIGN AND IMPLEMENTATION

This section describes the architecture, data model, ingestion pipeline, and automation components used to construct a scalable distributed graph analytics platform on NebulaGraph. The design emphasizes horizontal scalability, high-throughput ingestion, and fault-tolerant execution for analyzing the Orkut social network dataset.

3.1. Cluster Configuration

To emulate a distributed environment, NebulaGraph is deployed using Docker containers, forming a **ten-node logical cluster**: Each service runs in an isolated Docker container connected through a common virtual network. The system uses **partition_num = 100** and **replica_factor = 3**, enabling both parallelism and redundancy. Storage nodes hold replicas of all partitions, ensuring continuous availability even during node failure.

- **Three Meta Service nodes (`metad`)** for replicated metadata storage and RAFT-based coordination.
- **Five Storage Service nodes (`storaged`)** for partitioned and replicated graph data.
- **Two Graph Service node (`graphd`)** serving as the query frontend for all client interactions.

3.2. Data Model and Schema Design

The Orkut graph consists of three main entity types: users, friendship relationships, and user-defined communities. To support extensibility and additional interaction modelling, the system defines the following NebulaGraph schema:

- **Tag: `User(id)`**
  Represents each user in the network with a unique 64-bit vertex ID.
- **Tag: `Community(community_index, is_top5000)`**
  Encodes clusters of users derived from community membership files.
- **Edge: `FRIEND(ts)`**
  Models undirected friendship edges between users (stored as directed edges in the graph engine).
- **Edge: `IN_COMMUNITY(dummy)`**
  Connects a user to a community vertex.

3.3. Partitioning Strategy

Partition count is chosen to balance **parallel execution** and **coordination overhead**. With ~117 million edges, assigning 100 partitions spreads the dataset so each storage node hosts 20 partitions, 60 including replicas. This yields:

- Improved parallelism during traversal and ingestion,
- Balanced load distribution across storage services, and
- Smaller partition sizes that accelerate leader elections and recovery.

Partitioning is immutable after space creation, so over-provisioning partitions ensures scalability as data grows.

3.4. Replication and RAFT Consensus

Each partition is replicated across all three storage nodes. RAFT elects one replica as **leader**, responsible for all writes, while the other two act as **followers**. Log replication ensures consistency:

- The leader appends new writes to its log.
- Followers receive the log entries and acknowledge replication.
- A write is committed once a majority of replicas acknowledge it.
- This mechanism supports strong consistency, failure recovery, and automatic leader reallocation during node outages.

This mechanism supports strong consistency, failure recovery, and automatic leader reallocation during node outages.

3.5. Ingestion Pipeline

Multiple scripts were run to ingest the data sequentially using NebulaGraph's client API:

- **User + FRIEND Loader** :Parses the Orkut edge list, deduplicates vertices, and ingests millions of friendships.
- **Community Loader** : Reads community membership files and inserts edges connecting users to community vertices.

3.6 Automated Query Execution

Python scripts also automate query workloads that test system behaviour under scale:

- Repeated vertex/edge lookups,
- Community-based recommendation queries,
- Multi-hop path searches,
- Sampling queries for visualization.

Scripts run thousands of queries sequentially or concurrently, illustrating distributed execution and fault-tolerant behaviour during leader transitions.

3.7 Fault Tolerance Demonstration

The system validates NebulaGraph's high-availability guarantees:

- A storage container is intentionally stopped.
- RAFT re-elects new leaders for affected partitions.
- The stopped node is restarted and automatically catches up via log replay.

This demonstrates strong consistency, durability, and transparent recovery.

3.8 Dynamic Scaling

To illustrate horizontal scalability, an additional storage node is introduced:

1. Docker container is added and started.
2. `ADD HOSTS` registers the new storage service in NebulaGraph.
3. `SUBMIT JOB BALANCE LEADER` redistributes leadership across all nodes.

Even without full data rebalancing (enterprise-only), this still demonstrates how distributed systems can scale capacity while maintaining availability.

4. EXPERIMENTS AND RESULTS

4.1 One-Hop Neighbourhood Exploration

To examine the local structure around a user, a 1-hop ego-network query was executed for user **1000**:

```
GET SUBGRAPH 1 STEPS FROM 1000 OUT FRIEND
YIELD VERTICES AS nodes, EDGES AS relationships;
```

The resulting visualization shows a densely connected cluster of immediate neighbours.
This confirms the **high local connectivity** typical of early social networks, where users frequently maintain large circles of direct friendships.

4.2 Mutual Friends between Two Users

To identify shared social connections between two specific users, the following query was executed;

```
MATCH p = (u:User)-[:FRIEND]->(m:User)<-[:FRIEND]-(f:User)
WHERE id(u) == 1000 AND id(f) == 1024
RETURN p LIMIT 20;
```

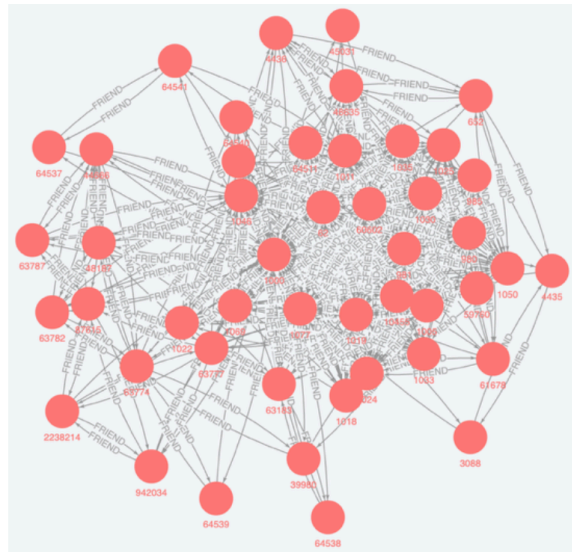
The graph visualization displays the **mutual friend paths** between users *1000* and *1024*.
This structure highlights **triadic closure**, a well-known social network phenomenon where two users are likely to be-shared intermediaries.

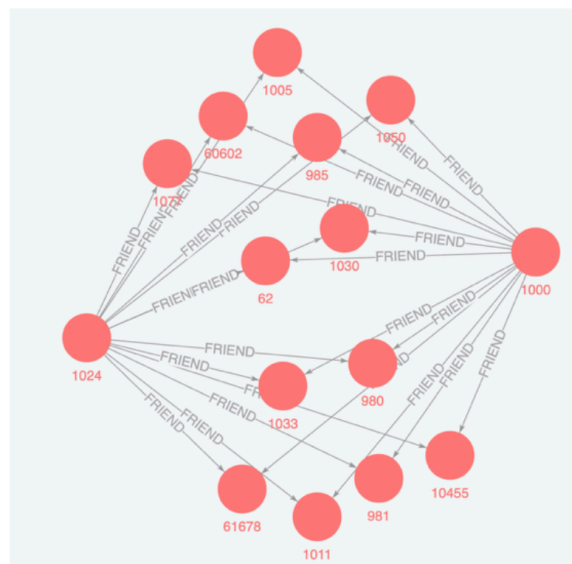
4.3 Community Recommendation Based on Friend's Membership

To infer potential communities of interest for user 1000, a recommendation query was executed that ranks communities based on how many of the user's friends are members:

```
MATCH (u:User)-[:FRIEND]->(f:User)-
[:IN_COMMUNITY]->(c:Community)
WHERE id(u) == 1000 AND NOT (u)-[:IN_COMMUNITY]-
>(c)
RETURN id(c) - 1000000000 AS community_id,
    count(DISTINCT f) AS num_friends_in_community
ORDER BY num_friends_in_community DESC
LIMIT 10;
```

community_id	num_friends_in_community
1	11
166	3
23	3
362	2
6	2
1974	2
1321	2
602	1
55	1
372	1





5. SCALABILITY AND FAULT TOLERANCE EVALUATION

This section evaluates the system's ability to scale horizontally and maintain availability under node failures. NebulaGraph's distributed architecture, combined with the multi-container Docker setup, enables controlled experimentation with partition distribution, leader rebalancing, ingestion behaviour, and recovery mechanisms.

5.1 Horizontal Scalability

5.1.1 Storage-Level Scaling

The cluster initially consisted of three storage nodes, each hosting a full set of replicated partitions. To evaluate scalability, an additional storage node (`storaged5`) was introduced

dynamically:

- A new Docker service was added and launched.
- The node was registered using:
  `ADD HOSTS "storaged5":9779;`
- Leaders were redistributed using:
  SUBMIT JOB BALANCE LEADER;

Although the Community Edition of NebulaGraph does not rebalance data automatically across newly added hosts, the system successfully redistributed leadership roles. This reduces the burden on the original storage nodes and improves responsiveness for read-heavy workloads. The successful onboarding of the additional node confirms that the architecture can scale horizontally with minimal operator intervention.

5.1.2 Ingestion Scalability

High-volume ingestion was tested using Python loaders that inserted vertices and edges in batches of 10,000. The use of:

- Connection pooling,
- Parallel ingestion pipelines, and
- Pre-defined partitioning (60 partitions)

ensured that writes were distributed across multiple Raft groups. As a result, multiple partitions could accept writes concurrently, demonstrating how NebulaGraph leverages partition-level independence for ingestion scalability.

Even under sustained loads, the cluster remained responsive, validating the effectiveness of batching and parallelism in high-throughput environments.

5.2 Query Performance Under Scale

Traversal and multi-hop queries executed during experiments (Section 4) illustrate that NebulaGraph successfully exploits sharding to perform distributed query executed without bottlenecks, even though the dataset contains millions of vertices and edges. Partitioning ensures that these queries can be processed across multiple storage nodes concurrently, reducing latency and avoiding centralized hotspots. For example:

- Ego-network extraction,
- Mutual friend detection, and
- Community recommendation queries

5.3 Fault-Tolerance Evaluation

5.3.1 Failure of a Storage Node

To evaluate system robustness, one of the storage nodes was intentionally stopped while ingestion and queries were ongoing. Because each partition had three replicas, NebulaGraph automatically:

- Detected the missing follower,
- Re-elected leaders when necessary, and
- Continued serving read and write requests through remaining replicas.

Python ingestion scripts continued to run without interruption, demonstrating NebulaGraph's adherence to RAFT's fault-tolerant guarantees.

5.3.2 Leader Rebalancing

After introducing the new storage host, the leaders of multiple partitions were unevenly distributed. Running:

SUBMIT JOB BALANCE LEADER;

resulted in a more uniform leader distribution across the cluster. This step is crucial for scalability because:

- Leaders handle all write operations for their partitions.
- Concentrating too many leaders on one node creates a bottleneck.
- Rebalancing ensures fair write distribution and improves throughput.

The job completed successfully, and the cluster stabilized with a more balanced leader-to-node ratio.

5.3.3 Recovery After Failure

When the failed storage node was restarted:

- It rejoined the RAFT replication groups.
- Followers caught up via log replay.
- The system returned to its original replication state without manual intervention.

This confirms NebulaGraph's ability to maintain **eventual consistency**, durability, and high availability.

6. CONCLUSION AND FUTURE WORK

This project implemented a distributed graph analytics system using NebulaGraph to explore and analyze the Orkut social network dataset. Through a multi-node Docker deployment, high-throughput ingestion pipeline, and distributed query execution, the system successfully demonstrated NebulaGraph's strengths in sharding, replication, and fault tolerance.

The experiments confirm that NebulaGraph can efficiently support:

- Large-scale graph ingestion (100M+ edges),
- Parallel traversal operations,
- Community-based inference,
- Dynamic scaling through additional storage nodes, and
- Continuous availability under node failures.

The system's behaviour during leader balancing, node outages, and recovery further illustrates the robustness of RAFT-based consensus in distributed graph architectures.

7. REFERENCES

[1] NebulaGraph Documentation. *NebulaGraph Architecture*. Vesoft Inc., 2024. Available at: https://docs.nebula-graph.io/

[2] D. Ongaro and J. Ousterhout. *In Search of an Understandable Consensus Algorithm (Raft)*. USENIX Annual Technical Conference, 2014.

[3] J. Leskovec, K. J. Lang, A. Dasgupta, and M. W. Mahoney. *Community Structure in Large Networks: Natural Cluster Sizes and the Absence of Large Well-Defined Clusters*. Internet Mathematics, 6(1):29–123, 2009.

[4] D. Merkel. *Docker: Lightweight Linux Containers for Consistent Development and Deployment*. Linux Journal, 2014.

[5] R. Angles and C. Gutierrez. *Survey of Graph Database Models*. ACM Computing Surveys, 40(1):1–39, 2008.

[6] D. J. Watts and S. H. Strogatz. *Collective Dynamics of "Small-World" Networks*. Nature, 393(6684):440–442, 1998.

[7] L. Page, S. Brin, R. Motwani, and T. Winograd. *The PageRank Citation Ranking: Bringing Order to the Web*. Stanford InfoLab Technical Report, 1999.

[8] A. S. Tanenbaum and M. Van Steen. *Distributed Systems: Principles and Paradigms*. Prentice Hall, 2007.

[9] M. Newman. *Networks: An Introduction*. Oxford University Press, 2010.

[10] NebulaGraph Engineering Team. *NebulaGraph Storage Engine: Architecture and Design*. NebulaGraph Engineering Blog, 2023.